An Efficient Agent Lookup Approach in Middlewares for Mobile Agents

Namiki Keita
Shibaura Institute of Technology
Graduate School of Engineering and Science
Email: l09070@shibaura-it.ac.jp

Hiroaki Fukuda
Shibaura Institute of Technology
Graduate School of Engineering and Science
Email: hiroaki@shibaura-it.ac.jp

Paul Leger
Universidad Católica del Norte
Email: pleger@ucn.cl

***Abstract*—A Wireless Sensor Network (WSN) is typically deployed on the place in which no electric source is provided, meaning that the battery consumption concern is crucial. Due to their deeply-embedded pervasive nature, applications running on WSNs need to adapt to the changes in physical environment or user preferences. In addition, developers of these applications must pay attention to a set of additional concerns such as limited hardware resources and the management of a set sensor nodes. To appropriately address these concerns, middlewares based on mobile agent based like Agilla have been proposed. Applications for these middlewares are executed by the communications among agents, thereby a common task is to lookup the agents. Although we can find several middlewares, none of current middlewares support the lookup of agents efficiently, implying in an unnecessary consumption of battery. In this paper, we propose an efficient lookup approach for mobile agent based middlewares in WSNs. Our proposal borrows ideas from CSN, a distributed hashtable algorithm for WSNs. We evaluate the advantages of our proposal through the comparison with a traditional lookup approach of agents in Agilla.**

I. Introduction

Wireless Sensor Networks (WSNs) [7] consist in a number of sensor nodes and have been used to detect events and/or collect data in various domains such as environment observation, structual health monitoring, inventory tracking, and smart homes [1]. Due to their embedded pervasive nature, each node is usually deployed on the place in which no electric source is provided and/or picking them up is not an easy task (*e.g.* inside of a forest). Therefore, each node is usually working by dry cell batteries, however, these nodes require to keep working for a long time (*e.g.* a few years [5]).

Developing an application for WSNs is difficult because developers need to pay attention to a network environment dynamically changing, the management of a set of sensor nodes, low-level instructions related to physical hardwares. Several middlewares for WSNs have been proposed to make a development easier [4], [5], [6]. It is possible to classify these middlewares into three types: (1) data oriented approaches that abstract a number of sensor nodes as one abstraction: WSN, (2) event-based approaches that provide callback methods, that are invoked when certain events are dispatched, and (3) mobile agent approaches, where their applications are based on the collaboration of mobile agents. In this paper, we focus on the mobile agent based approach because this approach is adequate when multiple applications are running on a WSN [4]. Agilla [4], a middleware for mobile agents for WSNs, allows an application to restrict itself to only reside on relevant nodes. As WSN environments change, this application can be self-adapted through the migration of its agents to the required locations. Moreover, an application in Agilla is executed by interactions among agents, meaning that an agent needs to know the exact location of other agents. Although Agilla allows agents to interact with other agents, this middleware (like similar proposals) does not provide functionalities to efficiently lookup the agent locations. As a result, an agent has to lookup the location of its target agent by ad-hoc manner, resulting in the consumption of a significant and unnecessary amount of the battery of sensor nodes.

In this paper, we propose an efficient lookup approach for mobile agent based middlewares in WSNs. Our proposal borrows ideas from CSN (Chord for WSNs) [8], a distributed hashtable algorithm. As a consequence of this efficient approach, it is possible to save battery consumptions of every node in a WSN. We implement our proposal on TinyOS, which uses the middleware Agilla and is one of the platform for WSNs. Finally, we show the advantages of our approach via a simulations that compare battery consumptions to an ad-hoc lookup method.

Paper roadmap. Section II briefly introduces Agilla and points out the battery consumption problem. Section III presents our approach to save battery consumption in the lookup of agents. Section IV describes the implementation of our proposal. Section V describes the assessment experiment and discusses the results obtained. Section VI concludes.

II. Agilla: A Middleware for WSNs

This section briefly describes Agilla. In addition, this section identifies the battery consumption problem.

A. Overview of Agilla

Agilla [4] is a well-known middleware for mobile agents on Wireless Sensors Networks (WSNs) [7]. Figure 1 shows the architecture of Agilla. Each node of Agilla has an interpreter to execute mobile agent programs. Developers can write a program using ISA (Instruction Set Architecture) prepared by Agilla. In addition, this middleware can also execute a number of agent programs on a node. The coordination among agents and accesses to physical resources are supported by a *tuple space* [2] which is so called a shared memory and located on each node and remotely accessible. By using these features, a program can restrict to reside only on relevant nodes and reactions to changes of the environment.

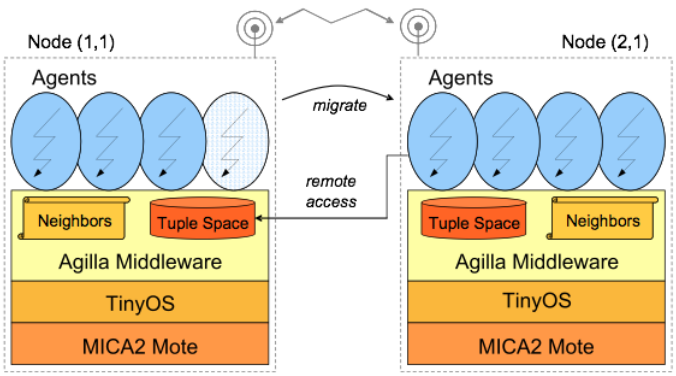

Fig. 1. Architecture of Agilla.

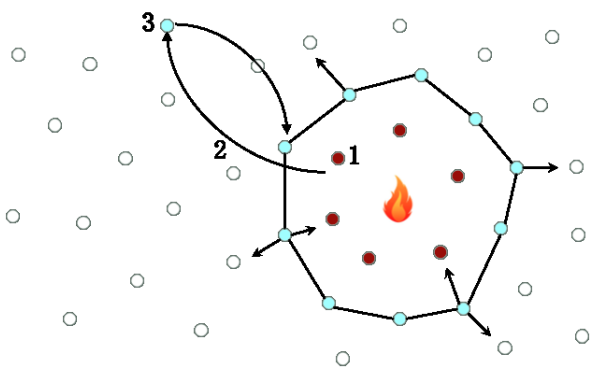

Fig. 2. An overview of the fire detection and tracking application.

B. Battery Consumption Problem

Using an example about the fire control, we introduce the battery consumption problem. Figure 2 shows an overview of the fire detection and tracking application in Agilla. When a fire breaks out, agents detect this fire (1) and send a message to a tracker agent (2). The tracker agent migrates to the fire and clone itself to form a perimeter. The perimeter is continuously adjusted based on the evolution of the fire. To send a message to a tracker agent, an agent must know the exact location of this agent tracker. Since Agilla (like similar proposals) does not provide methods to lookup the location of a agent on the fly, the location of a tracker agent is directly written in agents, meaning that a tracker agent should keep staying on a determined node. Otherwise, an agent needs to lookup the location of a tracker agent every time, resulting in consuming the battery of nodes very soon. Moreover, in the worst case, an agent could not find a tracker agent because Agilla agents can migrate at any time.

III. Applying our proposal to Lookup Agents

As we described in the previous section, Agilla does not provide an efficient method to lookup to an agent (that is moving). In this paper, we propose an efficient approach to lookup agents through a program location management system for mobile agent based middlewares. This approach is applied in Agilla. This section presents our approach with its required technologies.

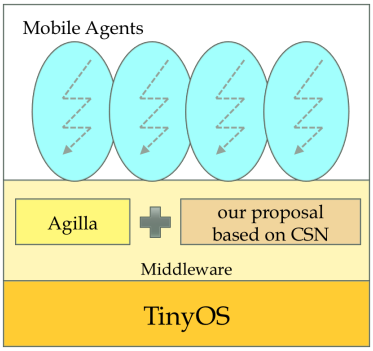

Fig. 3. Big picture of our proposal.

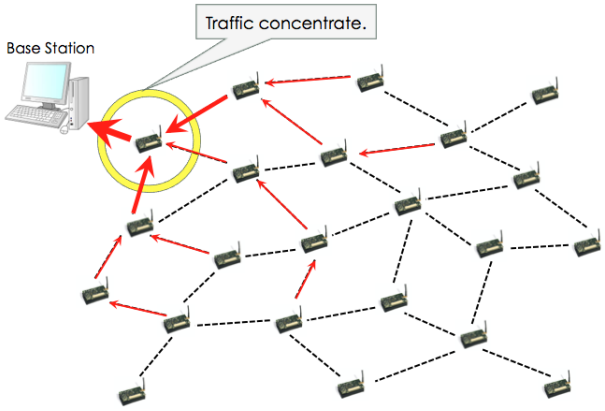

Fig. 4. Location Management at a base station.

An overview of our proposal. Figure 3 sums up our proposal. The figure shows that TinyOS uses the Agilla middleware to admin the interaction between mobile agents. As mentioned in the previous section, the agent lookup provided by Agilla searches in every sensor nodes to find an agent (and if the agent moves during the lookup, the wanted agent might not be found). We extended Agilla to support an more efficient lookup of agents. Our proposal uses a variation of the Chord algorithm [3], named CSN [8], which uses a distributed hash table to keep agent locations.

A. Location Agent Management

There are two ways to manage the locations of agents. First, managing the locations using a centralized manager at a base station, and the second one is using a distributed approach, meaning that every node in a WSN manages them partially. Now, we describe the advantages and drawbacks of both:

Centralized approach. This has the advantage that it is not so difficult to implement because an agent only uses a base station to lookup other agents. However, as shown in Figure 4, the traffic gets focused on the nodes around the base station. As a result, the battery consumption of the nodes around base station increases, resulting in that the lifetime of a WSN itself will be getting shorter[1].

[1] In a centralized approach, when the nodes around a base station consume a significant amount battery, the nodes cannot send any packets anymore, meaning that these nodes cannot send messages to a base station directly. Therefore, any node of a WSN cannot communicate with a base station anymore.

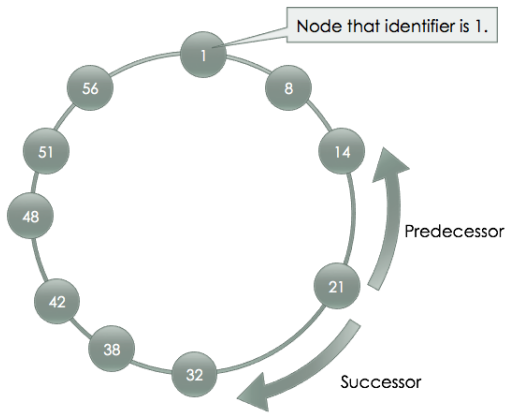

Fig. 5. Chord ring with 6 bits address space.

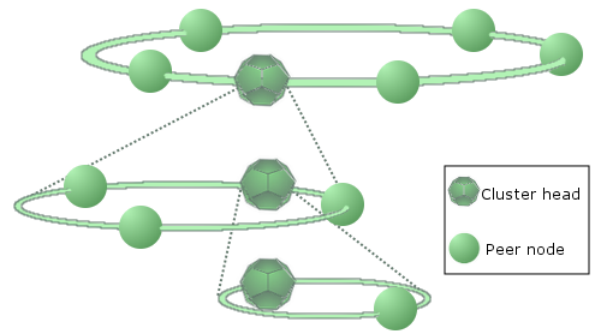

Fig. 6. Layering the ring.

Distributed approach. Although this approach is more difficult to implement than the centralized one, the load and battery consumptions are distributed among nodes of a WSN. Therefore, a WSN can be working more longer than that of the first approach. A longer lifetime of a WSN is crucial because a program is required to keep working to react an event (*e.g.* a change of the environment). In order to save battery, we adopt the distributed approach in WSNs.

The communication among nodes in WSNs is a kind of peer-to-peer (P2P). Hence, we adopt a Distributed Hash Table (DHT) [3], which has an efficient search for a distributed search approach. Although there are many kinds of DHT algorithms, we adopt Chord algorithm [3] that can search information faster [8]. However, we cannot directly apply the Chord algorithm because the original Chord algorithm does not care about the physical length among nodes. This is crucial because, in WSNs, a node can only communicate with the others inside of a radio wave. Therefore, we apply CSN (Chord for Sensor Network) [8] that adapts Chord for sensor networks to manage the agent locations. The next section, we briefly explain the Chord algorithm and its variant CSN.

B. Chord

Chord is a widely-known DHT algorithm. This algorithm utilizes an address space where the value of each identifier increases clockwise direction. In Chord, the value of each identifier is generated by a *SHA1* hash function, and the size of an address space is 160 bits because this function generates 160 bits number. Figure 5 shows an example of this address space, where the size is 6 bits with 10 nodes, whose identifiers are from 1 to 56, in a ring form. The distance between two nodes is defined by the clockwise subtractions of their given identifiers, therefore distance among nodes can be defined completely. For example in Figure 5, node 14 is close to node 1 compared to node 56[2].

In Chord, each target object (an agent in this context) is also assigned unique identifier by SHA1 and the location of which is managed by the closest node. For example in Figure 5, if an agent is assigned 30 as its identifier, the location of which is managed by node 32. Suppose a node, whose identifier is k receives a search request about another node with a identifier r. If r is greater and k and less than that of the successor of the node k, the successor of node k manages its target location. Otherwise node k forwards its request to the successor. Running over these procedures can finally resolve the target location. Besides, as each node also has a finger table that keeps a path to the node that is far by power of x order, Chord can provide $O(logN)$ search efficiency with this table (N is the total number of nodes in a ring).

1) CSN: As it is well-known that a packet loss usually happens when the distance between nodes is getting longer in WSNs, we cannot apply the original Chord algorithm to the location management of agents in a distributed manner. We have to consider the proximity network between nodes when a ring in Chord is created. CSN, a variant of Chord, is an algorithm that introduces the physical distance between nodes. CSN introduces a hierarchical clustering approach; each cluster in CSN is formed in a logical ring formation as the same way of the original Chord ring. Each CSN sensor node maintains a finger table of $O(logN)$ size about other nodes in a ring, and each node resolves all lookups by sending $O(logN)$ messages to other nodes. Besides, as shown in Figure 6, a super peer called cluster head is selected in each ring and each cluster head joins the ring of upper layer as a member node. CSN also keeps the number of nodes in each ring decreasing as the ring layer is going to the bottom. As a result, CSN guarantees the search efficiency as $O(MlogN)$[3].

We borrow the ideas behind the CSN algorithm to manage and lookup agents: a hierarchical clustering and connections of each cluster.

IV. Implementation

We implement our proposal on top of TinyOS [9] in the *nesC* language [10], an extension the C language for event-driven programming. TinyOS is a operating system provided for WSN's nodes. As shown in Figure 7, our proposal mainly consists in three components:

1) **PacketManager**. This includes a *Sender* component that decides it should allocate a process based on the received packets. This components adjusts the timing

[2]Notice this distance in Chord is independent from a physical distance among nodes.

[3]N is the number of nodes that joins the highest layer and M is the maximum path length of the energy efficient path between nodes + 1. See [8] more details.

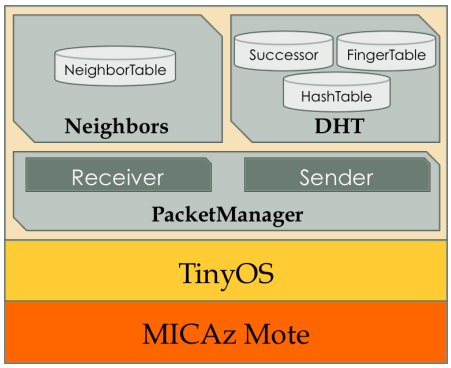

Fig. 7. Architecture of the program location management system.

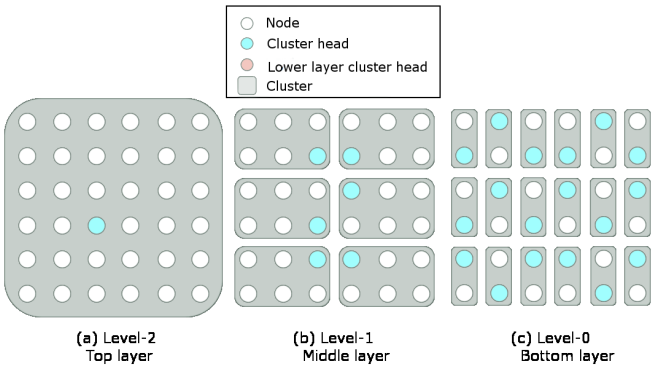

Fig. 8. Initial State of structuring DHT.

to send a packet in order to prevent interferences.

2) **Neighbors.** This collects packets and gathers the strength of each radio wave to make a neighbor table. We assume that the received packets from closer nodes have a stronger radio wave.

3) **DHT**. It controls the information of rings using DHT, meaning that it deals with the information of successor, predecessor, finger table, and hash table.

A. Structuring DHT Behavior

As mentioned in Section III, the distance between nodes is important in WSNs. Hence, we manually configure the clusters because the self-directive selection approach may not create appropriate clusters because of specific network restrictions. More precisely, we divide nodes into a set of clusters in each layer and assign a node to a cluster head in each cluster. We think it is feasible because, in WSNs, the number of nodes is fixed and each node cannot be added and/or removed from a general P2P network[4].

We show a initial state of 6 x 6 grid sensor nodes for structuring DHT in WSNs. In this case, the three layered rings are created: (a) an upper layer, (b) a middle layer and (c) a

[4]If the battery of a node gets empty, the topology changes. Although we now ignore this problem, we will apply other approaches such as a leader election algorithm to address this problem.

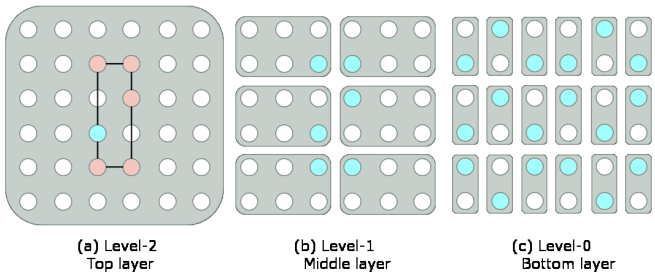

Fig. 9. After creating a ring of the upper layer.

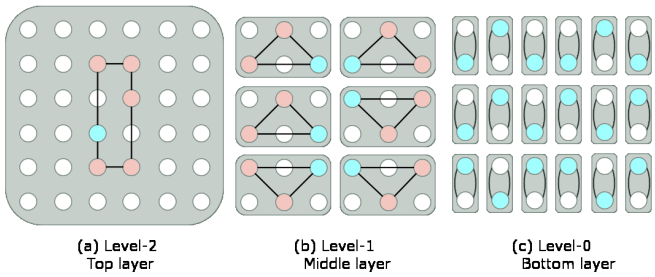

Fig. 10. After creating a ring of each layer.

bottom layer. In addition, as mentioned above, we specify each cluster head manually. For example in Figure 8 (a), we specify one cluster head and in Figure 8 (b), we create 6 cluster heads. Structuring DHT starts when the cluster head node of the upper layer receives a command:

1) Receive the command, the head node creates a neighbor table by using the *Neighbors* component. Note that the registered nodes with the table are also cluster heads in the next layers below.

2) A cluster head sends a command to create a ring to the nearest node and register it as a successor node. Then, when this node receives the command, this node registers the sender node as a predecessor.

3) The node, which is registered as a successor of the cluster head, sends an ACK to a sender node. This sender node starts selecting its successor. At the same time, by receiving an ACK, the cluster head registers its sender as a successor.

4) By repeating this procedure, a ring of the upper layer is created. We show the ring of the upper layer in Figure 10 (a).

After creating the ring of the upper layer, our proposal starts to create a ring of the middle layer as the same way. Then finally, our proposal ends up creating all rings in each layer.

B. Lookup Behavior

Figure 11 shows the procedure of the lookup behavior. We explain each step through the numbers in this Figure. A lookup always starts from a node of the upper layer:

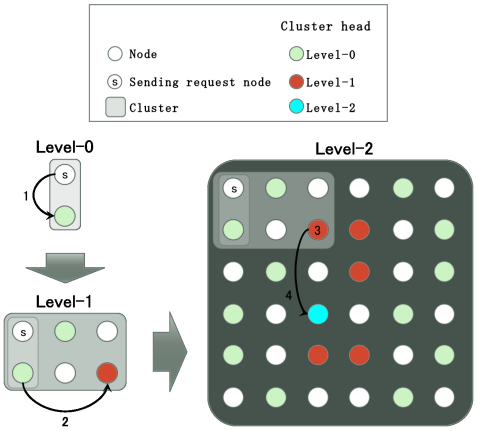

Fig. 11. Procedures of the lookup behavior.

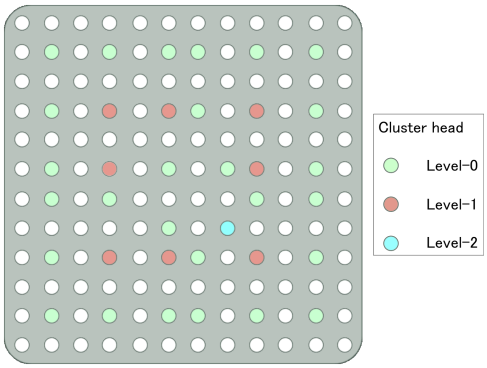

Fig. 12. The environment of simulations.

1) The node that starts to search sends a request, which includes a hash value of the target, to the cluster head node.

2) If the cluster does not belong to the upper layer, the request is forwarded to the cluster head node of the upper layer.

3) If the cluster is in the upper layer, the node in the upper layer compares the hash value included in a request to its own table.

4) If the node knows the location of the target, it returns the answer, otherwise it forwards the request to another node in the same layer or in the lower layer based on the comparison of the hash values.

V. Evaluation

This section shows the evaluation of our proposal by conducting an experiment that compares battery consumptions. To evaluate the battery consumptions of our proposal, we compare it to the *random walk* approach, the ad-hoc approach to lookup an agent in middlewares like Agilla.

TABLE I. The configuration of simulations.

	Layer	The number of node
The number of cluster	Level-2	1
	Level-1	9
	Level-0	36
Maximun number in a ring	Levle-2	9
	Level-1	4
	Level-0	2

TABLE II. The battery consumption of a program location management system.

	Consumed Power[J]
Structuring DHT	73.03
Per look-up	2.90

A. Experiment Setup

We used a simulator called PowerTOSSIMZ [12], which extends the well-known simulator for TinyOS called TOSSIM [11], to measure battery consumptions. Figure 12 and Table I show the simulation configuration. The simulation environment is a 12 x 12 grid network, *i.e.* 144 nodes. We evaluated the battery consumption in two stages: *a)* battery consumptions to build a DHT structure and *b)* per lookup operation.

B. Results and Discussion

Table II shows the battery consumption results of our proposal in the two stages previously mentioned. Instead, Table III shows the consumption per lookup operation with the random walk approach. As each node has 21,600J battery capacity, our proposal only consumes 0.338% of the battery of a node to create DHT structure. In addition, our proposal only consumes 0.013% of the battery for per lookup operation. In contrast, random walk approach consumes 0.033% of the battery for per lookup operation. Based on this result, we can appreciate that our proposal consumes some amount of the battery at the beginning, but our proposal after consumes less amount of the battery of the random walk approach. Figure 13 shows the relationship between battery consumptions and the number of lookup operations. Our approach requires additional battery consumption for structuring DHT, therefore, random walk approach consumes less battery when the number of lookup operation is less than 18 times. However, when the number of lookup operation is more than 18 times, our approach less consumes battery. We think this number (18 times) is enough small because WSNs are usually designed to work from one to a few years [5].

Furthermore, although the random walk approach can always find an agent in this experiment, ad-hoc approaches do not assure guarantee to complete an lookup within a specific period of time. Instead, our proposal can guarantee a successful search (*e.g.* location of an agent or false instead).

VI. Conclusion

As WSNs are exposed to a real world, many of their applications are required to adapt to the environment changes. In other words, these applications must be re-programmed quickly and easily. To address these problems, different middlewares for mobile agents have been proposed, where an application is composed of a set of agents and is executed

TABLE III. THE BATTERY CONSUMPTION OF THE RANDOM WALK.

	Consumed Power[J]
Per look-up	7.10

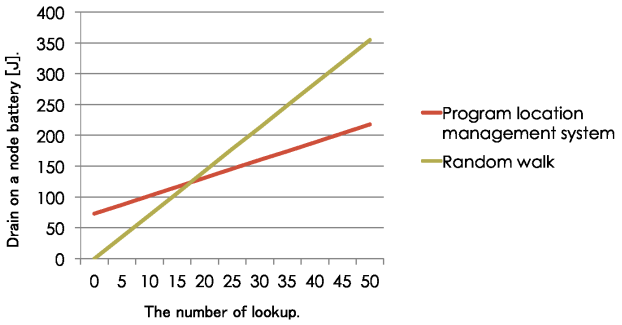

Fig. 13. Comparing our proposal to the random walk approach.

by the interactions of these agents. For this approach, an agent needs to know the exact location of its target agent beforehand. However, existing proposals, including Agilla, do not support an efficient lookup mechanism to lookup agents. In this paper, we propose an approach that borrows ideas from the CSN algorithm to efficiently lookup agents in WSNs within a specific period time. This efficient lookup allows WSN nodes to save battery consumptions. We implement our proposal on the TinyOS environment and verified its advantages via a comparison with a traditional lookup method.

Some extensions to our proposal are planned. For example, we currently have not implemented *finger table* [3] yet, meaning that our approach might show more efficient results if we use finger table. Therefore, we plan to implement finger table and conduct experienced to verify its advantage.

REFERENCES

[1] M. Satyanarayanan, Carnegie Mellon University.: Pervasive Computing: Vision andChallenges, In *Proceedings of the IEEE Personal Communications*, Vol.8, No.4, 2001, pp.10-17.

[2] D. Gelernter. : Generative Communication in Linda.: *ACM Transactions on Programming Languages and Systems*, Vol.7, No.1, 2001, pp.80-112.

[3] Ion Stoica, Robert Morris, David Karger, M. Frans Kaashoek, Hari Balakrishnan.: Chord: A scalable peer-to-peer lookup service for internet applications, In em SIGCOMM '01 Proceedings of the 2001 conference on Applications, technologies, architectures, and protocols for computer communications, 2001, pp.149-160.

[4] Chien-Liang Fok, Gruia-Catalin Roman, Chenyang Lu.: Agilla: A Mobile Agent Midleware for Self-Adaptive Wireless Sensor Networks, In *Proceedings of the ACM Transactions on Autonomous and Adaptive System*, Vol.4, No.3, 2009, Article 16.

[5] Samuel R. Madden, Michael J. Franklin, Joseph M. Hellerstein, Wei Hong.: TinyDB: an acquisitional query processing system for sensor networks, In *Proceedings of the ACM Transactions on Database Systems*, Vol.30, No.1, 2005, pp.122-173.

[6] T. Abdelzaher, B. Blum, Q. Cao, Y. Chen, D.Evans, J. George, S. George, L. Gu, T. He, S. Krishnamurthy, L. Luo, S. Son, J. Stankovic, R. Stoleru, A. Wood.: EnviroTrack: Towards an Environmental Computing Paradigm for Distributed Sensor Networks, In *Proceedings of the 24th International Conference on Distributed Computing System*, 2004, pp.582-589.

[7] Jennifer Yick, Biswanath Mukherjee, Dipak Ghosal.: Wireless sensor network survey, In *Journal of Computer networks*, Vol.52, No.12, 2008, pp.2292-2330.

[8] Muneeb Ali, Zartash Afzal Uzmi, LUMS.: CSN: A network protocol for serving dynamic queries in large-scale wireless sensor networks, In *Proceeding of the Second Annual Conference on Communication Networks and Services Research*, 2004, pp.165-174.

[9] Jason Hill, Robert Szewczyk, Alec Woo, Seth Hollar, David Culler, Kristofer Pister.: System architecture directions for networked sensors, In *Proceedings of the ACM SIGPLAN Notices*, Vol.35 No.11, 2000, pp.93-104.

[10] David Gay, Philip Levis, Robert von Behren, Matt Welsh, Eric Brewer, David Culler.: The nesC language: A holistic approach to networked embedded systems, In *Proceedings of the ACM SIGPLAN Notices*, Vol.38, No.5, 2003, pp.1-11.

[11] Philip Levis, Nelson Lee, Matt Welsh, David Culler.: TOSSIM: Accurate and scalable simulation of entire TinyOS applications, In *Proceedings of the First ACM Conference on Embedded Networked Sensor Systems*, 2003, pp.126-137.

[12] Enrico Perla, Art Ó Catháin, Ricardo Simon Carbajo, Meriel Huggard, Ciarán Mc Goldrick.: PowerTOSSIM z: realistic energy modelling for wireless sensor network environments, In *Proceedings of the 3nd ACM workshop on Performance monitoring and measurement of heterogeneous wireless and networks*, 2008, pp.35-42.